UNIVERSIDAD
Panamericana

PROGRAMACIÓN AVANZADA

Sesión 06

Práctica de Bit Manipulation

→ NOTAS DE CLASE

Universidad Panamericana
Dr. Miguel Alcaraz Rivera • malcaraz@up.edu.mx

2026

Índice

1 Técnicas fundamentales **2**

2 Máscaras de *k* bits bajos **2**

3 Ejercicios **3**
3.1 Bloque A — Calentamiento 3
3.2 Bloque B — Implementación 3
3.3 Bloque C — Programación competitiva 4

4 Resumen de técnicas **6**

5 Metacognición **6**

Técnicas fundamentales

La sesión anterior cubrió los operadores de bits. Esta sesión los **aplicamos**. Antes de pasar a los ejercicios, hay cuatro operaciones que aparecen en casi todo código que manipula bits. Todas giran en torno a la misma idea: `1 « k` es una **máscara de un bit** — solo el bit k encendido.

Operación	Expresión	Qué hace
Leer bit k	`(x » k) & 1`	Devuelve 1 si el bit k está encendido, 0 si no
Encender bit k	`x | (1 « k)`	Pone el bit k en 1 sin tocar los demás
Apagar bit k	`x & ~(1 « k)`	Pone el bit k en 0 sin tocar los demás
Invertir bit k	`x ^(1 « k)`	Cambia el bit k: 0 $\rightarrow$ 1, 1 $\rightarrow$ 0

✔ **Recuerda**

¿Por qué funcionan?

- OR con `1 « k` **fuerza** el bit k a 1 (OR con 1 siempre da 1).
- AND con `~(1 « k)` **fuerza** el bit k a 0 (AND con 0 siempre da 0).
- XOR con `1 « k` **invierte** solo el bit k (XOR con 1 cambia el valor; con 0 lo deja igual).

Operación	Resultado	Binario
`x` (original)	180	`10110100`
leer bit 1: `(x»1)&1`	0	(el bit 1 está apagado)
encender bit 1: `x|(1«1)`	182	`10110110`
apagar bit 2: `x&~(1«2)`	176	`10110000`
invertir bit 7: `x^(1«7)`	52	`00110100`

Máscaras de *k* bits bajos

Una operación frecuente es generar una máscara con los k bits más bajos encendidos y todos los demás apagados:

k	Máscara	Valor
3	`00000111`	7
5	`00011111`	31
8	`11111111`	255

La expresión es `(1 « k) - 1`.

¿Por qué funciona? `1 « k` tiene exactamente el bit k encendido (vale 2^k). Si le restamos 1, ese bit se apaga y todos los bits por debajo de él se encienden — exactamente la máscara que necesitamos.

Combinar esta máscara con AND extrae los k bits bajos de cualquier valor: `x & ((1 « k) - 1)` equivale a `x % (2^k)`, pero sin división.

Ejercicios

• Bloque A — Calentamiento

Evalúa cada expresión **a mano**, mostrando los 8 bits de cada operando en cada paso. Luego verifica con el cuaderno.

1. `17 & 12`

$$17 = \texttt{????????}, \quad 12 = \texttt{????????}, \quad \texttt{17 \& 12} = \texttt{????????} = ?$$

2. `7 « 2`

$$7 = \texttt{????????}, \quad \texttt{7 « 2} = \texttt{????????} = ?$$

3. `13 | 12`

$$13 = \texttt{????????}, \quad 12 = \texttt{????????}, \quad \texttt{13 | 12} = \texttt{????????} = ?$$

4. `(~7 | (1 « 1) | (1 « 3) | (1 « 5)) & 29`

Pista: calcula ~7 primero (8 bits bajos), luego aplica cada OR, y al final el AND con 29.

5. `(~(13 » 2)) & 39`

Este es el ejemplo resuelto en la sesión 05 — inténtalo sin mirar.

• Bloque B — Implementación

6. Escribe cuatro funciones en C++ que reciban `x` y `k`, y apliquen cada técnica de la sección 1. Pruébalas con `x = 0b11001010` (202) y estos casos:

Operación	Caso	Esperado
Leer bit k	$k = 3$	`1`
Apagar bit k	$k = 6$	`10001010` = 138
Encender bit k	$k = 0$	`11001011` = 203
Invertir bit k	$k = 7$	`01001010` = 74

7. Escribe una función `mascara(k)` que devuelva un entero con los k bits más bajos encendidos, usando exactamente una expresión de bits. Verifica con $k \in \{3, 5, 8\}$.

• Bloque C — Programación competitiva

Los siguientes problemas tienen el formato de un juez en línea: entrada, salida y restricciones. La solución debe ser eficiente: **sin bucles donde baste una expresión de bits**.

8. ¿Es potencia de 2?

Entrada: un entero n con $1 \leq n \leq 10^9$.
Salida: 1 si n es potencia de 2, 0 si no.

Entrada	Salida
1	1
4	1
7	0
16	1
100	0

Pista: una potencia de 2 tiene exactamente un bit encendido. ¿Qué hace `n & (n-1)` con ese único bit?

9. Cuenta bits encendidos (popcount).

Entrada: un entero n con $0 \leq n \leq 10^9$.
Salida: cantidad de bits en 1 en la representación binaria de n.

Entrada	Salida
0	0
7	3
13	3
255	8

Pista A (bucle simple): usa `n & 1` para leer el bit bajo y `n >>= 1` para avanzar.
Pista B (truco rápido): `n & (n-1)` apaga el bit encendido de menor peso. Cuenta cuántas veces puedes repetirlo hasta llegar a 0.

10. El elemento único.

Entrada: un arreglo donde todos los elementos aparecen **exactamente dos veces**, excepto uno.
Salida: el elemento que aparece una sola vez.
Restricción: tiempo $O(n)$, espacio $O(1)$ — sin arreglos auxiliares.

Arreglo	Salida
{3, 5, 3, 7, 5}	7
{1, 4, 1, 2, 4, 9, 2}	9

Pista: recuerda que `x ^ x = 0` y `x ^ 0 = x`. ¿Qué pasa si aplicas XOR a todos los elementos del arreglo?

11. Intercambio sin variable temporal.

Intercambia los valores de `a` y `b` usando **solo XOR**, sin declarar ninguna variable auxiliar.

Pista: aplica XOR tres veces. Traza qué contiene cada variable después de cada paso.

12. Distancia de Hamming.

La **distancia de Hamming** entre dos enteros es el número de posiciones de bit donde difieren.

Entrada: dos enteros a, b con $0 \leq a, b \leq 10^9$.
Salida: su distancia de Hamming.

Entrada	Salida
$a = 29,\ b = 15$	2
$a = 1,\ b = 4$	2

$29 =$ `11101`, $15 =$ `01111`, XOR $=$ `10010` — 2 bits distintos.

Pista: combina XOR con el popcount del problema anterior.

13. El bit encendido más bajo.

Entrada: un entero $n > 0$.
Salida: el **valor** del bit encendido de menor peso (el `1` más a la derecha).

Entrada	Salida
12 (`1100`)	4
24 (`11000`)	8
7 (`0111`)	1
40 (`101000`)	8

Pista: en complemento a 2, `-n` invierte todos los bits y suma 1. ¿Qué produce `n & -n`?

14. Paridad.

Entrada: un entero $n \geq 0$.
Salida: `0` si la cantidad de bits en `1` es par, `1` si es impar.

Entrada	Salida
7 (`111`)	1
6 (`110`)	0
255 (`11111111`)	0
13 (`1101`)	1

Pista: la paridad es el XOR de todos los bits del número. Puedes acumular con `res ^= (n & 1)` y desplazar.

15. ★ **Contar subconjuntos (bitmask).**

Una **máscara de bits** de k bits representa todos los subconjuntos posibles de un conjunto de k elementos: cada bit i indica si el elemento i está incluido.

Tarea: dado k, imprime todas las máscaras de 0 a $2^k - 1$, mostrando qué elementos incluye cada una.

Para $k = 3$ (elementos {A, B, C}):

```
000  ->  {}
001  ->  {A}
010  ->  {B}
011  ->  {A, B}
100  ->  {C}
101  ->  {A, C}
110  ->  {B, C}
111  ->  {A, B, C}
```

Pista: itera `mask` de 0 a `(1«k)-1`. Para cada `mask`, verifica cada bit con `(mask » i) & 1`.

Resumen de técnicas

✔ **Recuerda**

Técnica	Expresión	Uso en competencia
Leer bit k	`(x » k) & 1`	Verificar si un elemento está en subconj
Encender bit k	`x | (1 « k)`	Agregar elemento a subconjunto
Apagar bit k	`x & ~(1 « k)`	Quitar elemento de subconjunto
Invertir bit k	`x ^(1 « k)`	Toggle de estado
Máscara k bits	`(1 « k) - 1`	Extraer los k bits bajos
¿Potencia de 2?	`n > 0 && !(n & (n-1))`	Verificar en $O(1)$
Bit más bajo	`n & -n`	Iterar sobre bits encendidos
Apagar bit más bajo	`n & (n-1)`	Popcount eficiente
Elemento único	XOR de todo el arreglo	Problema clásico $O(n)/O(1)$
Distancia de Hamming	`popcount(a ^b)`	Códigos de error, similitud
Iterar subconjuntos	`for mask in 0..(1«k)-1`	DP sobre subconjuntos

Metacognición

Escribe un **correo electrónico** a tu papá, mamá o tutor contándole sobre los temas que hemos visto en clase hasta el momento.

El correo debe incluir:

- **Qué sabías** del tema antes de esta clase.
- **Qué no sabías** y aprendiste.

- **Qué te gustó** de la actividad.
- **Cómo piensas aplicarlo** en tu vida profesional.

Escríbelo como si le estuvieras *contando* a esa persona, con tus propias palabras, no como un reporte técnico. Pon al profesor en copia (**cc**).